Homework 7

All functions should be in a main.cpp file in the HW/Homework07 folder of your GitHub class repository. Only include: `<iostream>`, `<string>`, `<vector>`, `<cstdlib>`, `<cctype>`, `"Map.h"`, and `"util.h"`. Add the following lines:

```
using namespace std;
using namespace dshw;
template <typename K, typename V>
using Map = MapOpen<K, V>;
// using Map = MapChain<K, V>;
```

Task 1: (4 points)

Create a function with the following header: `Map<char, int> frequencyCount(const string& str)`
The function should return a Map that contains the frequency count of each character in the input string. The function should ignore spaces and be case-insensitive (i.e., 'A' and 'a' should be counted as the same character). Use the isSpace() function from `"util.h"` to check for spaces.

```
frequencyCount("hello world") => {'H': 1, 'E': 1, 'L': 3, 'O': 2, 'W': 1, 'R': 1, 'D': 1}
frequencyCount("follower") => {'F': 1, 'E': 1, 'L': 2, 'O': 2, 'W': 1, 'R': 1}
```

Task 2: (4 points)

Create a function with the following header: `unsigned long long fibonacci(unsigned int n)`
The function should return the nth Fibonacci number using memoization to optimize performance. Memoization is a technique where you store the results of expensive function calls and reuse them when the same inputs occur again. Use a Map to store the Fibonacci numbers that have already been computed. Declare the Map as static so that it persists throughout all function calls. The Fibonacci sequence is defined as follows:

$$F(0) = 0$$
$$F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2) \text{ for } n > 1$$

```
fibonacci(10) => 55; (0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + 21 + 34)
```

Task 3: (4 points)

Create a function with the following header: `unsigned long long romanToInt(const string& s)`
The function should convert a Roman numeral string to its integer equivalent. Roman numerals are represented by the following symbols: `{{'I',1},{'V',5},{'X',10},{'L',50},{'C',100},{'D',500},{'M',1000}}`. Use a Map to store the Roman numeral symbols and their corresponding integer values. You can use the getRomanNumeral() function from `"util.h"` to get generate test numbers. The rules for converting Roman numerals to integers are:

Working from right to left:	Working from left to right
If a numeral is < the previous (right) it is subtracted from the total. Else, the numeral is added to the total.	If a numeral is >= the next (right), it is added to the total. Else, the numeral is subtracted from the total.

```
romanToInt("MCMXCIV") => 1994. (1000 - 100 + 1000 - 10 + 100 - 1 + 5)
romanToInt("LIV") => 54. (50 - 1 + 5)
```

Task 4: (4 points)

Create a function with the following header:

```
double gpaCalculator(const vector<Pair<string, double>>& grades)
```

The function should calculate the GPA based on a vector of Pair objects, where each Pair contains a letter grade and the number of credits for a course. Use the following mapping of letter grades to grade points: `{{"A+",4.0},{"A",4.0},{"A-",3.7},{"B+",3.3},{"B",3.0},{"B-",2.7},{"C+",2.3},{"C",2.0},{"C-",1.7},{"D+",1.3},{"D",1.0},{"F",0.0}}`. Use a Map to store the letter grade to grade point mapping. The getKey() method of the Pair class should be used to access the letter grade, and the getValue() method should be used to access the number of credits from the parameter. The GPA is calculated as the total grade points from the vector parameter's letter grades divided by the total credits from the vector parameter's course credits. Round the GPA to at most three decimal places using the decimalPlaces() function from `"util.h"`. Pass the calculated GPA and 3 as arguments to the function.

`gpaCalculator({{"A", 3.0},{"B+", 4.0},{"C", 3.0}}) => 3.12`

Total grade points = $(4.0 \times 3.0) + (3.3 \times 4.0) + (2.0 \times 3.0) = 12.0 + 13.2 + 6.0 = 31.2$
Total credits = $3.0 + 4.0 + 3.0 = 10.0$
GPA = Total grade points / Total credits = 31.2 / 10.0 = 3.12

`gpaCalculator({{"A+",4.0},{"A",3.0},{"B-",3.0},{"C+",2.0},{"F",3.0}}) => 2.713`

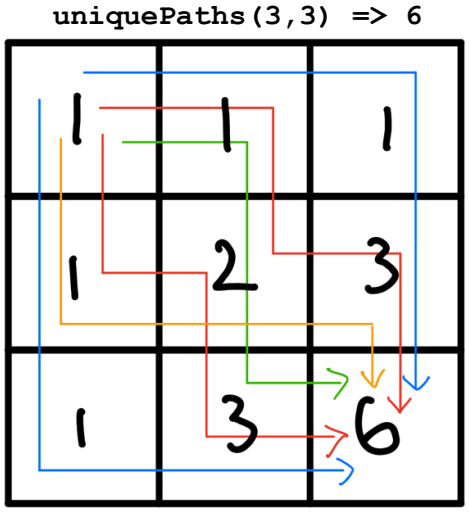
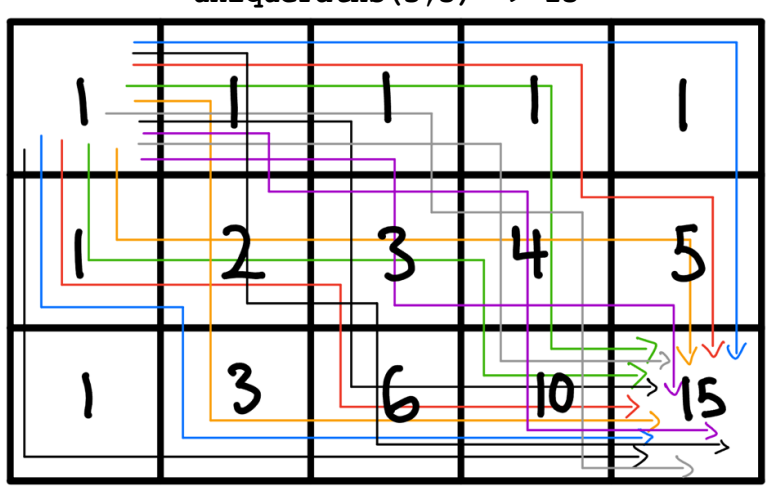
Total grade points = $(4.0 \times 4.0) + (4.0 \times 3.0) + (2.7 \times 3.0) + (2.3 \times 2.0) + (0.0 \times 3.0) = 16.0 + 12.0 + 8.1 + 4.6 + 0.0 = 40.7$
Total credits = $4.0 + 3.0 + 3.0 + 2.0 + 3.0 = 15.0$
GPA = Total grade points / Total credits = 40.7 / 15.0 = 2.713333 = 2.713

Task 5: (4 points)

Total Create a function with the following header: `int uniquePaths(int m, int n)`
The function should return the number of unique paths from the top-left corner to the bottom-right corner of an m x n grid. You can only move either down or right at any point in time. The parameters m and n represent the number of rows and columns in the grid, respectively. The number of unique paths for a given cell can be calculated as the sum of the unique paths from the cell directly above it and the cell directly to the left of it. Use memoization to optimize performance by storing the results of previously computed paths in a static Map. Use a Pair<int, int> as the key in the Map to represent the current position in the grid. Use the drawGrid() function from `"util.h"` to visualize the grid and the number of unique paths for testing cases. Use the following approach:

1. If m == 1 or n == 1, return 1 (base case).
2. Check if the result for the current (m, n) is already in the Map. If it is, return the stored result.
3. Otherwise, compute the result by recursively calling uniquePaths(m-1, n) and uniquePaths(m, n-1), storing it in the Map, then returning the result.

`uniquePaths(3,3) => 6`



`uniquePaths(3,5) => 15`